

Grundlagen der Programmierung:

8. Objektorientierung I

Tutoren:

Daniel Grillmeyer, Ludwig Friedl, Jan Kerber, Jennifer Muck

1. Objektorientierung – Teil I
 1. Instanzvariablen/-attribute
 2. Konstruktoren
 3. Instanzmethoden
2. Besprechung von Aufgabenblatt 1

OBJEKTORIENTIERUNG

Die Tiere

Wir wollen folgende Tiere simulieren:



Eisbaer



Katze



Dromedar



Hund



Pinguin

Die Tiere

Wir wollen folgende Tiere simulieren:



Eisbaer



Katze



Dromedar



Hund



Pinguin

- Erstellen Sie für Katze, Hund und ein beliebiges weiteres Tier eine eigene Klasse, sowie eine Klasse Main inkl. main-Methode, in der wir unsere Klassen testen können

INSTANZVARIABLEN

Instanzvariablen

- In den Klassen lassen sich Attribute festlegen, die jede Instanz dieser Klasse haben soll
- In unserem Beispiel wollen wir, dass jedes Tier einen Namen bekommt

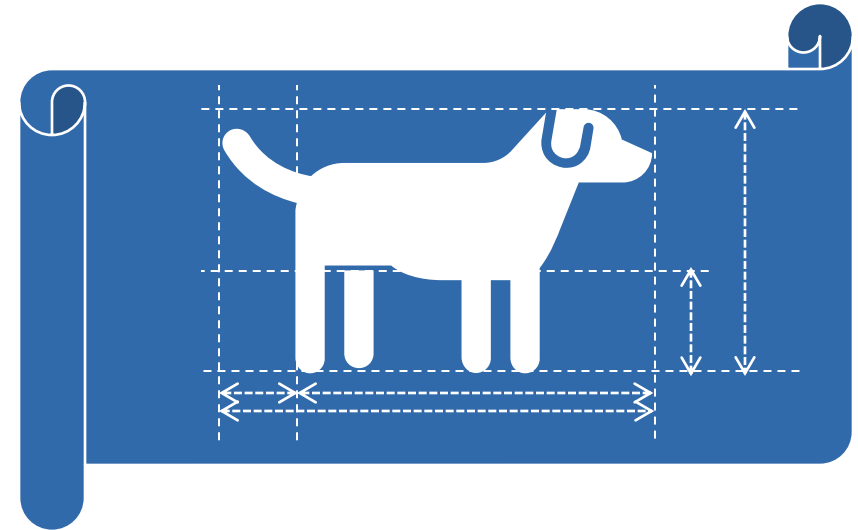
Aufgaben:

- Geben Sie jeder Tierart das Attribut **name** vom Typ String
- Erstellen Sie in der main-Methode **einen Hund namens „Bello“** und **eine Katze namens „Kitty“**
- Geben Sie anschließend die Namen der Tiere auf der Konsole aus

➤ Musterlösung

```
1 Hund einHund = new Hund();
2 einHund.name = "Bello";
3
4 Katze eineKatze = new Katze();
5 eineKatze.name = "Kitty";
6
7 System.out.println(einHund.name);
8 System.out.println(eineKatze.name);
```


KONSTRUKTOREN



Ein Tier konstruieren

- Klassen erlauben es uns ein **Objekt** bzw. eine **Instanz** einer Klasse zu erzeugen
- Bisher benutzten wir `new` `Hund()`; um ein neues **Objekt** bzw. eine **Instanz** von der Klasse `Hund` zu erstellen
- Da wir keinen eigenen Konstruktor für die Klasse `Hund` definiert haben wird der **Standard-Konstruktor** bzw. **Default-Konstruktor** aufgerufen, den jede Klasse automatisch besitzt (solange kein eigener Konstruktor definiert wird)
- Wir wollen allerdings nicht den Standard-Konstruktor verwenden, sondern einen eigenen Konstruktor definieren, der gleich den Namen des Hundes setzt
- Dafür bekommt der Konstruktor ein Argument vom Typ `String`:

```
Hund(String name) {...}
```
- Erstellen Sie einen Konstruktor für die Klasse `Hund`, der dem Attribut `name` einen Wert zuweist

Ein Tier konstruieren

Mögliche Lösung

```
1 Hund(String neuerName) {  
2     name = neuerName;  
3 }
```

Besser (Konvention):

```
1 Hund(String name) {  
2     this.name = name;  
3 }
```

- Das Argument name hat hier den gleichen Bezeichner wie das Attribut **name**
- Deswegen wird zur Unterscheidung das Attribut mit **this.name** aufgerufen

Puzzler: Konstruktoren

- Welche Zuweisungen sind erlaubt?

```
1 Hund wuffi = new Hund("Wuffi");  
2 Hund keks = Hund("Keks");  
3 Hund fido = new Hund();  
4 fido.name = "Fido";
```

1

2

3 & 4

Puzzler: Konstruktoren

- Welche Zuweisungen sind erlaubt?

```
1 Hund wuffi = new Hund("Wuffi");  
2 Hund keks = Hund("Keks");  
3 Hund fido = new Hund();  
4 fido.name = "Fido";
```

1

2

3 & 4

Puzzler: Konstruktoren

- Welche Zuweisungen sind erlaubt?

```
1 Hund wuffi = new Hund("Wuffi");  
2 Hund keks = Hund("Keks");  
3 Hund fido = new Hund();  
4 fido.name = "Fido";
```

1

2

3 & 4

Puzzler: Konstruktoren

- Welche Zuweisungen sind erlaubt?

```
1 Hund wuffi = new Hund("Wuffi");  
2 Hund keks = Hund("Keks");  
3 Hund fido = new Hund();  
4 fido.name = "Fido";
```

1

2

3 & 4

Warum?

Puzzler: Konstruktoren

- Welche Zuweisungen sind erlaubt?

```
1 Hund wuffi = new Hund("Wuffi");  
2 Hund keks = Hund("Keks");  
3 Hund fido = new Hund();  
4 fido.name = "Fido";
```

1

2

3 & 4

Warum?

- Der Konstruktor muss immer mit **new** aufgerufen werden

Puzzler: Konstruktoren

- Welche Zuweisungen sind erlaubt?

```
1 Hund wuffi = new Hund("Wuffi");  
2 Hund keks = Hund("Keks");  
3 Hund fido = new Hund();  
4 fido.name = "Fido";
```

1

2

3 & 4

Warum?

- Der Konstruktor muss immer mit **new** aufgerufen werden

Puzzler: Konstruktoren

- Welche Zuweisungen sind erlaubt?

```
1 Hund wuffi = new Hund("Wuffi");  
2 Hund keks = Hund("Keks");  
3 Hund fido = new Hund();  
4 fido.name = "Fido";
```

1

2

3 & 4

Warum?

- Der Konstruktor muss immer mit **new** aufgerufen werden
- Sobald ein eigener Konstruktor definiert wird, ist der Standard-Konstruktor nicht mehr verfügbar

Konstruktor überladen

- Wenn wir den Standard-Konstruktor zusätzlich zu unserem neuen Konstruktor benutzen wollen, müssen wir ihn wieder explizit hinzufügen

```
1 public class Hund {  
2  
3     public String name = "";  
4  
5     Hund(String name) {  
6         this.name = name;  
7     }  
8  
9  
10  
11  
12 }
```

Konstruktor überladen

- Wenn wir den Standard-Konstruktor zusätzlich zu unserem neuen Konstruktor benutzen wollen, müssen wir ihn wieder explizit hinzufügen

```
1 public class Hund {  
2  
3     public String name = "";  
4  
5     Hund(String name) {  
6         this.name = name;  
7     }  
8  
9     Hund() {  
10    }  
11  
12 }
```

Konstruktor überladen

- Wenn wir den Standard-Konstruktor zusätzlich zu unserem neuen Konstruktor benutzen wollen, müssen wir ihn wieder explizit hinzufügen

```
1 public class Hund {  
2  
3     public String name = "";  
4  
5     Hund(String name) {  
6         this.name = name;  
7     }  
8  
9     Hund() {  
10    }  
11  
12 }
```

- Wenn mehrere Konstruktoren mit verschiedenen Argumenten definiert werden spricht man von Überladen bzw. Overloading
- Ähnlich auch bei Methoden möglich

INSTANZMETHODEN

Fütterungszeit

- Instanzmethoden sind Methoden, die zu einer bestimmten Instanz gehören und auf einer Instanz aufgerufen werden
 - Im Gegensatz zu statischen Methoden, die auf Klassen aufgerufen werden
 - Damit unsere Hunde nicht verhungern, bekommen sie die Fähigkeit zu fressen
 - Dafür implementieren wir eine Methode `friss()`, die bei Aufruf einen Text in der Form „<Name>: *mampf*“ ausgibt
- Implementieren Sie die Methode `friss()` (ohne Rückgabewert) in der Klasse `Hund`!

Musterlösung:

```
1 public void friss() {  
2     System.out.println(name + ": *mampf*");  
3 }
```

- Ein Vorteil gegenüber statischen Methoden ist hier, dass die Instanzvariable **name** aufgerufen werden kann
- Um anderen Tieren auch die Methode `friss()` zu geben müssten wir sie im Copy & Paste-Verfahren in die anderen Klassen kopieren. Eleganter ist hier die Verwendung von **Vererbung**

To Be Continued

LÖSUNG SCHRIFTLICHES AUFGABENBLATT 1

Lösung Aufgabenblatt 1

- **Nicht Teil des Skripts – wird nur in der Übung besprochen**